

Adding functionality to placeholder types

Document #: P3171R0

Date: 2024-03-04

Project: Programming Language C++

Audience: LEWG

Reply-to: Peter Dimov

<pdimov@gmail.com>

Barry Revzin

<barry.revzin@gmail.com>

Contents

[1 Introduction](#)

[2 Proposal](#)

[2.1 Dealing with &x](#)

[2.2 Additional Details](#)

[2.3 Implementation Experience](#)

[2.4 Wording](#)

[2.5 Feature-Test Macro](#)

[3 References](#)

§ 1 Introduction

As noted in [P2760R0], there are a lot of function objects for operators in the standard library, but several operators are missing. This paper proposes to add the functionality for all the missing operators, but to also do it in a different way than simply by adding function objects.

[Boost.Lambda2] is a Boost library (written by Peter) which makes it possible to write very terse, simple operations, by building upon the `std::bind` machinery. When Barry was implementing `std::views::zip` [P2321R2], a range adaptor whose implementation requires forwarding various operators across a tuple, Boost.Lambda2 provided a very nice way to implement those operations. Here is a comparison between a hand-written lambda solution, function objects, and the placeholder solution that Lambda2 offers:

Handwritten Lambdas	Named Function Objects	Boost.Lambda2
<pre>auto operator*() const { return tuple_transform(current_, [](auto& it) -> decltype(auto) { return *it; }); }</pre>	<pre>auto operator*() const { return tuple_transform(current_, dereference{}); }</pre>	<pre>auto operator*() const { return tuple_transform(current_, *_1); }</pre>
<pre>auto operator++() -> iterator& { tuple_for_each(current_, [](auto& it) { ++it; }); return *this; }</pre>	<pre>auto operator++() -> iterator& { tuple_for_each(current_, prefix_increment{}); return *this; }</pre>	<pre>auto operator++() -> iterator& { tuple_for_each(current_, ++_1); return *this; }</pre>

It’s not just that the Lambda2 alternatives are overwhelmingly terser (it’s very hard to beat 3 characters for the dereference operation, especially compared to the handwritten lambda that must use `-> decltype(auto)` and is thus 46 characters long), they more directly express exactly the work being done.

Lambda2 also offers a more expressive way of doing common predicates, even in the case where the named function object already exists. Let's take an example where you want to write a predicate for if the argument is negative (an example Barry previously wrote about on his blog [here](#)), there are several ways to do it:

```
// hand-written lambda (28 characters)
[](auto e) { return e < 0; }

// attempting to use std::less{} (19 characters, but... uh...)
bind(less{}, _1, 0)

// Boost.Lambda2 (6 characters)
_1 < 0
```

It also allows for an approach to address the question of projections. Let's say that rather than finding a negative number, we want to find a Point whose x coordinate is negative:

```
// hand-written lambda (30 characters)
find_if(points, [](Point p){ return p.x < 0; })

// Boost.Lambda 2 (18 characters)
find_if(points, _1->* &Point::x < 0)
```

Or if the x coordinate is 0:

```
// hand-written lambda (31 characters)
find_if(points, [](Point p){ return p.x == 0; })

// using projection (12 characters, but cryptic)
find(points, 0, &Point::x);

// Boost.Lambda 2 (19 characters)
find_if(points, _1->* &Point::x == 0)
```

Note that this latter usage could be improved significantly with something like [\[P0060R0\]](#), which would actually allow for writing the predicate `_1.x == 0`. Which is difficult to beat.

You can see more examples in the [\[Boost.Lambda2\]](#) docs.

§ 2 Proposal

We propose to solve the issue of missing operator function objects in the standard library, as well as less-than-ergonomic lambda syntax for common predicates, by standardizing Boost.Lambda2. That is not a large proposal. The standard library already provides placeholders, `std::placeholders::_1` and friends. The standard library also already provides `std::bind`, which is already implemented in a way that supports composition of bind expressions. All we need to do is add operators.

We additionally add the missing operator function objects. Now, most of the missing operator function objects and placeholder operators are easy enough to add, except one: taking an object's address.

§ 2.1 Dealing with `&x`

Now, this particular operator has two problems. First, making `&_1` work requires overload unary `operator&()` and that seems particularly questionable, even in cases like this. And in order to make this broadly useful, we couldn't just overload it as a member function, it'd have to be a non-member - to support things like `&*_1` or any other combination of operations (which is part of the value of Lambda2). That's a bit too much code for having `&x` not actually mean address-of.

We could potentially address this problem by adding in a function like `std::placeholders::addr(x)` to mean `addressof`, so that instead of the cute `&_1` syntax you'd have to write `addr(_1)`, which doesn't have any issues with `&`. Note that we cannot call this function `addressof` because while `addressof(_1)` would be okay, `addressof(addressof(_1))` would become ambiguous (unless we also change `std::addressof`, as we're about to discuss).

Second, the obvious name for a function object taking the address of an object would be `std::addressof` - but that already exists, as a function template. We cannot change `std::addressof` to be a type - that would break all code that uses it. We could potentially

change it to be an object - that would break only ADL uses of it, but given the nature of `std::addressof` those seem pretty unlikely to be common, so it's potentially a feasible route to take. It would also allow `_1->*std::addressof` (in the absence of `addr(_1)` or similar formulation) as a short-ish way of expressing this.

For now, we're going to punt on both problems and simply not support either a terse `addressof` on placeholders or providing an `addressof` function object.

§ 2.2 Additional Details

Boost.Lambda2 additionally provides two helper function objects: `first` and `second`, such that `_1->*first` gives you the first element of the type (as by `std::get<0>`) and `_1->*second` gives you the second. This is done by just providing function objects that perform these operations, similar to the proposed `get_key` and `get_value` [P2769R1].

Also, while most operators take forwarding references, there are two additional overloads of `>>` and `<<` which are special-cased such that operations like `std::cout << _1` work and capture `std::cout` by reference. The special-casing is necessary because otherwise `std::cout` would be captured by value, which is not allowed, and users would have to write `std::ref(std::cout) << _1`.

§ 2.3 Implementation Experience

Has been shipping in Boost since 1.77 (August 2021).

§ 2.4 Wording

Extend 22.10.2 [functional.syn] to add the additional function objects:

```
namespace std {

    // ...

    // [bitwise.operations], bitwise operations
    template<class T = void> struct bit_and; // freestanding
    template<class T = void> struct bit_or; // freestanding
    template<class T = void> struct bit_xor; // freestanding
    template<class T = void> struct bit_not; // freestanding
    template<> struct bit_and<void>; // freestanding
    template<> struct bit_or<void>; // freestanding
    template<> struct bit_xor<void>; // freestanding
    template<> struct bit_not<void>; // freestanding

+ // [additional.operations], additional transparent operations
+ struct subscript; // freestanding
+ struct left_shift; // freestanding
+ struct right_shift; // freestanding
+ struct unary_plus; // freestanding
+ struct dereference; // freestanding
+ struct increment; // freestanding
+ struct decrement; // freestanding
+ struct postfix_increment; // freestanding
+ struct postfix_decrement; // freestanding
+
+ // [compound.operations], compound assignment operations
+ struct plus_equal; // freestanding
+ struct minus_equal; // freestanding
+ struct multiplies_equal; // freestanding
+ struct divides_equal; // freestanding
+ struct modulus_equal; // freestanding
+ struct bit_and_equal; // freestanding
+ struct bit_or_equal; // freestanding
+ struct bit_xor_equal; // freestanding
+ struct left_shift_equal; // freestanding
+ struct right_shift_equal; // freestanding

    // ...
}
```

```
}
```

Extend 22.10.2 [\[functional.syn\]](#) to add operators:

```
namespace std {
    // ...
    namespace placeholders {
        // M is the implementation-defined number of placeholders
        see below _1; // freestanding
        see below _2; // freestanding
        .
        .
        .
        see below _M; // freestanding
+   template<class A, class B> constexpr auto operator+(A&&, B&&); // freestanding
+   template<class A, class B> constexpr auto operator-(A&&, B&&); // freestanding
+   template<class A, class B> constexpr auto operator*(A&&, B&&); // freestanding
+   template<class A, class B> constexpr auto operator/(A&&, B&&); // freestanding
+   template<class A, class B> constexpr auto operator%(A&&, B&&); // freestanding
+   template<class A> constexpr auto operator-(A&&); // freestanding
+
+   template<class A, class B> constexpr auto operator==(A&&, B&&); // freestanding
+   template<class A, class B> constexpr auto operator!=(A&&, B&&); // freestanding
+   template<class A, class B> constexpr auto operator<(A&&, B&&); // freestanding
+   template<class A, class B> constexpr auto operator>(A&&, B&&); // freestanding
+   template<class A, class B> constexpr auto operator<=(A&&, B&&); // freestanding
+   template<class A, class B> constexpr auto operator>=(A&&, B&&); // freestanding
+   template<class A, class B> constexpr auto operator<=>(A&&, B&&); // freestanding
+
+   template<class A, class B> constexpr auto operator&&(A&&, B&&); // freestanding
+   template<class A, class B> constexpr auto operator|| (A&&, B&&); // freestanding
+   template<class A> constexpr auto operator!(A&&); // freestanding
+
+   template<class A, class B> constexpr auto operator&(A&&, B&&); // freestanding
+   template<class A, class B> constexpr auto operator|(A&&, B&&); // freestanding
+   template<class A, class B> constexpr auto operator^(A&&, B&&); // freestanding
+   template<class A> constexpr auto operator~(A&&); // freestanding
+
+   template<class A, class B> constexpr auto operator<<(A&&, B&&); // freestanding
+   template<class A, class B> constexpr auto operator<<=(A&, B&&); // freestanding
+
+   template<class A, class B> constexpr auto operator>>(A&&, B&&); // freestanding
+   template<class A, class B> constexpr auto operator>>=(A&, B&&); // freestanding
+
+   template<class A> constexpr auto operator+(A&&); // freestanding
+   template<class A> constexpr auto operator*(A&&); // freestanding
+   template<class A> constexpr auto operator++(A&&); // freestanding
+   template<class A> constexpr auto operator--(A&&); // freestanding
+   template<class A> constexpr auto operator++(A&&, int); // freestanding
+   template<class A> constexpr auto operator--(A&&, int); // freestanding
+
+   template<class A, class B> constexpr auto operator+=(A&&, B&&); // freestanding
+   template<class A, class B> constexpr auto operator-=(A&&, B&&); // freestanding
+   template<class A, class B> constexpr auto operator*=(A&&, B&&); // freestanding
+   template<class A, class B> constexpr auto operator/=(A&&, B&&); // freestanding
+   template<class A, class B> constexpr auto operator%=(A&&, B&&); // freestanding
+   template<class A, class B> constexpr auto operator&=(A&&, B&&); // freestanding
+   template<class A, class B> constexpr auto operator|=(A&&, B&&); // freestanding
+   template<class A, class B> constexpr auto operator^=(A&&, B&&); // freestanding
+   template<class A, class B> constexpr auto operator<<=(A&&, B&&); // freestanding
+   template<class A, class B> constexpr auto operator>>=(A&&, B&&); // freestanding
+
+   template<class A, class B> constexpr auto operator->*(A&&, B&&); // freestanding
+
+   inline constexpr unspecified first = unspecified; // freestanding
+   inline constexpr unspecified second = unspecified; // freestanding
    }
```

```
// ...  
}
```

Add two new sections after 22.10.11 [\[bitwise.operations\]](#):

§ Additional operations [additional.operations]

§ Class subscript [additional.operations.subscript]

```
struct subscript {  
    template<class T, class U> constexpr auto operator()(T&& t, U&& u) const  
        -> decltype(std::forward<T>(t)[std::forward<U>(u)]);  
  
    using is_transparent = unspecified;  
};  
  
template<class T, class U> constexpr auto operator()(T&& t, U&& u) const  
    -> decltype(std::forward<T>(t)[std::forward<U>(u)]);
```

1 Returns: `std::forward<T>(t)[std::forward<U>(u)]`.

§ Class left_shift [additional.operations.left_shift]

```
struct left_shift {  
    template<class T, class U> constexpr auto operator()(T&& t, U&& u) const  
        -> decltype(std::forward<T>(t) << std::forward<U>(u));  
  
    using is_transparent = unspecified;  
};  
  
template<class T, class U> constexpr auto operator()(T&& t, U&& u) const  
    -> decltype(std::forward<T>(t) << std::forward<U>(u));
```

1 Returns: `std::forward<T>(t) << std::forward<U>(u)`.

§ Class right_shift [additional.operations.right_shift]

```
struct right_shift {  
    template<class T, class U> constexpr auto operator()(T&& t, U&& u) const  
        -> decltype(std::forward<T>(t) >> std::forward<U>(u));  
  
    using is_transparent = unspecified;  
};  
  
template<class T, class U> constexpr auto operator()(T&& t, U&& u) const  
    -> decltype(std::forward<T>(t) >> std::forward<U>(u));
```

1 Returns: `std::forward<T>(t) >> std::forward<U>(u)`.

§ Class unary_plus [additional.operations.unary_plus]

```
struct unary_plus {  
    template<class T> constexpr auto operator()(T&& t) const  
        -> decltype(+std::forward<T>(t));  
  
    using is_transparent = unspecified;  
};  
  
template<class T> constexpr auto operator()(T&& t) const  
    -> decltype(+std::forward<T>(t));
```

1 Returns: `+std::forward<T>(t)`.

§ Class dereference [additional.operations.dereference]

```
struct dereference {  
    template<class T> constexpr auto operator()(T&& t) const
```

```

    -> decltype(*std::forward<T>(t));

    using is_transparent = unspecified;
};

template<class T> constexpr auto operator()(T&& t) const
    -> decltype(*std::forward<T>(t));

```

1 *Returns: *std::forward<T>(t).*

§ Class increment [additional.operations.increment]

```

struct increment {
    template<class T> constexpr auto operator()(T&& t) const
        -> decltype(++std::forward<T>(t));

    using is_transparent = unspecified;
};

template<class T> constexpr auto operator()(T&& t) const
    -> decltype(++std::forward<T>(t));

```

1 *Returns: ++std::forward<T>(t).*

§ Class decrement [additional.operations.decrement]

```

struct decrement {
    template<class T> constexpr auto operator()(T&& t) const
        -> decltype(--std::forward<T>(t));

    using is_transparent = unspecified;
};

template<class T> constexpr auto operator()(T&& t) const
    -> decltype(--std::forward<T>(t));

```

1 *Returns: --std::forward<T>(t).*

§ Class postfix_increment [additional.operations.postfix_increment]

```

struct postfix_increment {
    template<class T> constexpr auto operator()(T&& t) const
        -> decltype(std::forward<T>(t)++);

    using is_transparent = unspecified;
};

template<class T> constexpr auto operator()(T&& t) const
    -> decltype(std::forward<T>(t)++);

```

1 *Returns: std::forward<T>(t)++.*

§ Class postfix_decrement [additional.operations.postfix_decrement]

```

struct postfix_decrement {
    template<class T> constexpr auto operator()(T&& t) const
        -> decltype(std::forward<T>(t)--);

    using is_transparent = unspecified;
};

template<class T> constexpr auto operator()(T&& t) const
    -> decltype(std::forward<T>(t)--);

```

1 *Returns: std::forward<T>(t)--.*

§ Compound assignment operations [compound.operations]

§ Class `plus_equal` [`compound.operations.plus_equal`]

```
struct plus_equal {
    template<class T, class U> constexpr auto operator()(T&& t, U&& u) const
        -> decltype(std::forward<T>(t) += std::forward<U>(u));

    using is_transparent = unspecified;
};

template<class T, class U> constexpr auto operator()(T&& t, U&& u) const
    -> decltype(std::forward<T>(t) += std::forward<U>(u));
```

1 *Returns:* `std::forward<T>(t) += std::forward<U>(u)`.

§ Class `minus_equal` [`compound.operations.minus_equal`]

```
struct minus_equal {
    template<class T, class U> constexpr auto operator()(T&& t, U&& u) const
        -> decltype(std::forward<T>(t) -= std::forward<U>(u));

    using is_transparent = unspecified;
};

template<class T, class U> constexpr auto operator()(T&& t, U&& u) const
    -> decltype(std::forward<T>(t) -= std::forward<U>(u));
```

1 *Returns:* `std::forward<T>(t) -= std::forward<U>(u)`.

§ Class `multiplies_equal` [`compound.operations.multiplies_equal`]

```
struct multiplies_equal {
    template<class T, class U> constexpr auto operator()(T&& t, U&& u) const
        -> decltype(std::forward<T>(t) *= std::forward<U>(u));

    using is_transparent = unspecified;
};

template<class T, class U> constexpr auto operator()(T&& t, U&& u) const
    -> decltype(std::forward<T>(t) *= std::forward<U>(u));
```

1 *Returns:* `std::forward<T>(t) *= std::forward<U>(u)`.

§ Class `divides_equal` [`compound.operations.divides_equal`]

```
struct divides_equal {
    template<class T, class U> constexpr auto operator()(T&& t, U&& u) const
        -> decltype(std::forward<T>(t) /= std::forward<U>(u));

    using is_transparent = unspecified;
};

template<class T, class U> constexpr auto operator()(T&& t, U&& u) const
    -> decltype(std::forward<T>(t) /= std::forward<U>(u));
```

1 *Returns:* `std::forward<T>(t) /= std::forward<U>(u)`.

§ Class `modulus_equal` [`compound.operations.modulus_equal`]

```
struct modulus_equal {
    template<class T, class U> constexpr auto operator()(T&& t, U&& u) const
        -> decltype(std::forward<T>(t) %= std::forward<U>(u));

    using is_transparent = unspecified;
};

template<class T, class U> constexpr auto operator()(T&& t, U&& u) const
    -> decltype(std::forward<T>(t) %= std::forward<U>(u));
```

1 *Returns:* `std::forward<T>(t) %= std::forward<U>(u)`.

§ Class `bit_and_equal` [`compound.operations.bit_and_equal`]

```
struct bit_and_equal {
    template<class T, class U> constexpr auto operator()(T&& t, U&& u) const
        -> decltype(std::forward<T>(t) &= std::forward<U>(u));

    using is_transparent = unspecified;
};

template<class T, class U> constexpr auto operator()(T&& t, U&& u) const
    -> decltype(std::forward<T>(t) &= std::forward<U>(u));
```

1 *Returns:* `std::forward<T>(t) &= std::forward<U>(u)`.

§ Class `bit_or_equal` [`compound.operations.bit_or_equal`]

```
struct bit_or_equal {
    template<class T, class U> constexpr auto operator()(T&& t, U&& u) const
        -> decltype(std::forward<T>(t) |= std::forward<U>(u));

    using is_transparent = unspecified;
};

template<class T, class U> constexpr auto operator()(T&& t, U&& u) const
    -> decltype(std::forward<T>(t) |= std::forward<U>(u));
```

1 *Returns:* `std::forward<T>(t) |= std::forward<U>(u)`.

§ Class `bit_xor_equal` [`compound.operations.bit_xor_equal`]

```
struct bit_xor_equal {
    template<class T, class U> constexpr auto operator()(T&& t, U&& u) const
        -> decltype(std::forward<T>(t) ^= std::forward<U>(u));

    using is_transparent = unspecified;
};

template<class T, class U> constexpr auto operator()(T&& t, U&& u) const
    -> decltype(std::forward<T>(t) ^= std::forward<U>(u));
```

1 *Returns:* `std::forward<T>(t) ^= std::forward<U>(u)`.

§ Class `left_shift_equal` [`compound.operations.left_shift_equal`]

```
struct left_shift_equal {
    template<class T, class U> constexpr auto operator()(T&& t, U&& u) const
        -> decltype(std::forward<T>(t) <<= std::forward<U>(u));

    using is_transparent = unspecified;
};

template<class T, class U> constexpr auto operator()(T&& t, U&& u) const
    -> decltype(std::forward<T>(t) <<= std::forward<U>(u));
```

1 *Returns:* `std::forward<T>(t) <<= std::forward<U>(u)`.

§ Class `right_shift_equal` [`compound.operations.right_shift_equal`]

```
struct right_shift_equal {
    template<class T, class U> constexpr auto operator()(T&& t, U&& u) const
        -> decltype(std::forward<T>(t) >>= std::forward<U>(u));

    using is_transparent = unspecified;
};
```



```
template<class T, class U> constexpr auto operator()(T&& t, U&& u) const
-> decltype(std::forward<T>(t) >>= std::forward<U>(u));
```

1 Returns: `std::forward<T>(t) >>= std::forward<U>(u)`.

Extend 22.10.15.5 [func.bind.place]:

```
namespace std::placeholders {
    // M is the number of placeholders
+ template <int J>
+ struct placeholder { // exposition only
+     template <class... Args>
+         constexpr decltype(auto) operator()(Args&&... ) const noexcept;
+     template <class T>
+         constexpr auto operator[](T&& ) const;
+ };

    see below _1;
    see below _2;
    .
    .
    .
    see below _M;
}
```

1 The number M of placeholders is implementation-defined.

2 All placeholder types meet the *Cpp17DefaultConstructible* and *Cpp17CopyConstructible* requirements, and their default constructors and copy/move constructors are constexpr functions that do not throw exceptions. It is implementation-defined whether placeholder types meet the *Cpp17CopyAssignable* requirements, but if so, their copy assignment operators are constexpr functions that do not throw exceptions.

3 Placeholders should be defined as:

```
- inline constexpr unspecified _1{};
+ inline constexpr placeholder<1> _1{};
```

If they are not, they are declared as:

```
- extern unspecified _1;
+ extern placeholder<1> _1;
```

4 Placeholders are freestanding items ([freestanding.item]).

```
template <int J>
template <class... Args>
decltype(auto) placeholder<J>::operator()(Args&&... args) const noexcept;
```

5 Constraints: `sizeof...(Args) >= J` is true.

6 Returns: `std::forward<Args>(args)...[J - 1]`.

```
template <int J>
template <class T>
auto placeholder<J>::operator[](T&& t) const;
```

7 Returns: `bind(subscript(), *this, std::forward<T>(t))`.

8 Each operator function declared in this clause is constrained on at least one of the parameters having a type T which satisfies `is_placeholder_v<remove_cvref_t<T>> || is_bind_expression_v<remove_cvref_t<T>>` is true.

```
template<class A, class B> constexpr auto operator+(A&& a, B&& b);
```

9 Returns: `bind(plus<>(), std::forward<A>(a), std::forward<B>(b))`.

```
template<class A, class B> constexpr auto operator-(A&& a, B&& b);
```

```
10 Returns: bind(minus<>(), std::forward<A>(a), std::forward<B>(b)).
    template<class A, class B> constexpr auto operator*(A&& a, B&& b);

11 Returns: bind(multiplies<>(), std::forward<A>(a), std::forward<B>(b)).
    template<class A, class B> constexpr auto operator/(A&& a, B&& b);

12 Returns: bind(divides<>(), std::forward<A>(a), std::forward<B>(b)).
    template<class A, class B> constexpr auto operator%(A&& a, B&& b);

13 Returns: bind(modulus<>(), std::forward<A>(a), std::forward<B>(b)).
    template<class A> constexpr auto operator-(A&& a);

14 Returns: bind(negate<>(), std::forward<A>(a)).
    template<class A, class B> constexpr auto operator==(A&& a, B&& b);

15 Returns: bind(equal_to<>(), std::forward<A>(a), std::forward<B>(b)).
    template<class A, class B> constexpr auto operator!=(A&& a, B&& b);

16 Returns: bind(not_equal_to<>(), std::forward<A>(a), std::forward<B>(b)).
    template<class A, class B> constexpr auto operator<(A&& a, B&& b);

17 Returns: bind(less<>(), std::forward<A>(a), std::forward<B>(b)).
    template<class A, class B> constexpr auto operator>(A&& a, B&& b);

18 Returns: bind(greater<>(), std::forward<A>(a), std::forward<B>(b)).
    template<class A, class B> constexpr auto operator<=(A&& a, B&& b);

19 Returns: bind(less_equal<>(), std::forward<A>(a), std::forward<B>(b)).
    template<class A, class B> constexpr auto operator>=(A&& a, B&& b);

20 Returns: bind(greater_equal<>(), std::forward<A>(a), std::forward<B>(b)).
    template<class A, class B> constexpr auto operator<==(A&& a, B&& b);

21 Returns: bind(compare_three_way(), std::forward<A>(a), std::forward<B>(b)).
    template<class A, class B> constexpr auto operator&&(A&& a, B&& b);

22 Returns: bind(logical_and<>(), std::forward<A>(a), std::forward<B>(b)).
    template<class A, class B> constexpr auto operator||(A&& a, B&& b);

23 Returns: bind(logical_or<>(), std::forward<A>(a), std::forward<B>(b)).
    template<class A> constexpr auto operator!(A&& a);

24 Returns: bind(logical_not<>(), std::forward<A>(a)).
    template<class A, class B> constexpr auto operator&(A&& a, B&& b);

25 Returns: bind(bit_and<>(), std::forward<A>(a), std::forward<B>(b)).
    template<class A, class B> constexpr auto operator|(A&& a, B&& b);

26 Returns: bind(bit_or<>(), std::forward<A>(a), std::forward<B>(b)).
    template<class A, class B> constexpr auto operator^(A&& a, B&& b);
```

```

27 Returns: bind(bit_xor<>(), std::forward<A>(a), std::forward<B>(b)).
    template<class A> constexpr auto operator~(A&& a);

28 Returns: bind(bit_not<>(), std::forward<A>(a)).
    template<class A, class B> constexpr auto operator<<(A&& a, B&& b);

29 Constraints: is_base_of_v<ios_base, remove_cvref_t<A>> is false.

30 Returns: bind(left_shift(), std::forward<A>(a), std::forward<B>(b)).
    template<class A, class B> constexpr auto operator<<(A& a, B&& b);

31 Constraints: is_base_of_v<ios_base, remove_cvref_t<A>> is true.

32 Returns: bind(left_shift(), ref(a), std::forward<B>(b)).

33 Remarks: This overload allows expressions like std::cout << _1 << '\n' to work.
    template<class A, class B> constexpr auto operator>>(A&& a, B&& b);

34 Constraints: is_base_of_v<ios_base, remove_cvref_t<A>> is false.

35 Returns: bind(right_shift(), std::forward<A>(a), std::forward<B>(b)).
    template<class A, class B> constexpr auto operator>>(A& a, B&& b);

36 Constraints: is_base_of_v<ios_base, remove_cvref_t<A>> is true.

37 Returns: bind(right_shift(), ref(a), std::forward<B>(b)).
    template<class A> constexpr auto operator+(A&& a);

38 Returns: bind(unary_plus(), std::forward<A>(a)).
    template<class A> constexpr auto operator*(A&& a);

39 Returns: bind(dereference(), std::forward<A>(a)).
    template<class A> constexpr auto operator++(A&& a);

40 Returns: bind(increment(), std::forward<A>(a)).
    template<class A> constexpr auto operator--(A&& a);

41 Returns: bind(decrement(), std::forward<A>(a)).
    template<class A> constexpr auto operator++(A&& a, int);

42 Returns: bind(postfix_increment(), std::forward<A>(a)).
    template<class A> constexpr auto operator--(A&& a, int);

43 Returns: bind(postfix_decrement(), std::forward<A>(a)).
    template<class A, class B> constexpr auto operator+=(A&& a, B&& b);

44 Returns: bind(plus_equal(), std::forward<A>(a), std::forward<B>(b)).
    template<class A, class B> constexpr auto operator--=(A&& a, B&& b);

45 Returns: bind(minus_equal(), std::forward<A>(a), std::forward<B>(b)).
    template<class A, class B> constexpr auto operator*=(A&& a, B&& b);

46 Returns: bind(multiplies_equal(), std::forward<A>(a), std::forward<B>(b)).

```

```

template<class A, class B> constexpr auto operator/=(A&& a, B&& b);
47 Returns: bind(divides_equal(), std::forward<A>(a), std::forward<B>(b)).

template<class A, class B> constexpr auto operator%=(A&& a, B&& b);
48 Returns: bind(modulus_equal(), std::forward<A>(a), std::forward<B>(b)).

template<class A, class B> constexpr auto operator&=(A&& a, B&& b);
49 Returns: bind(bit_and_equal(), std::forward<A>(a), std::forward<B>(b)).

template<class A, class B> constexpr auto operator|=(A&& a, B&& b);
50 Returns: bind(bit_or_equal(), std::forward<A>(a), std::forward<B>(b)).

template<class A, class B> constexpr auto operator^=(A&& a, B&& b);
51 Returns: bind(bit_xor_equal(), std::forward<A>(a), std::forward<B>(b)).

template<class A, class B> constexpr auto operator<=(A&& a, B&& b);
52 Returns: bind(left_shift_equal(), std::forward<A>(a), std::forward<B>(b)).

template<class A, class B> constexpr auto operator>=(A&& a, B&& b);
53 Returns: bind(right_shift_equal(), std::forward<A>(a), std::forward<B>(b)).

template<class A, class B> constexpr auto operator--*(A&& a, B&& b);
54 Returns: bind(std::forward<B>(b), std::forward<A>(a)).

55 The name first denotes a customization point object ([customization.point.object]). Given a subexpression E:
(55.1) — If E has class or enumeration type and get<0>(E) is a valid expression where the meaning of get is established by
performing argument-dependent lookup only ([basic.lookup.argdep]), then first(E) is expression-equivalent to
get<0>(E).
(55.2) — Otherwise, first(E) is ill-formed.

56 The name second denotes a customization point object ([customization.point.object]). Given a subexpression E:
(56.1) — If E has class or enumeration type and get<1>(E) is a valid expression where the meaning of get is established by
performing argument-dependent lookup only ([basic.lookup.argdep]), then second(E) is expression-equivalent to
get<1>(E).
(56.2) — Otherwise, second(E) is ill-formed.

```

§ 2.5 Feature-Test Macro

Add an entry to 17.3.2 [\[version.syn\]](#) for this

```
+ #define __cpp_lib_placeholder_operators 2024XXL // freestanding, also in <functional>
```

§ 3 References

[Boost.Lambda2] Peter Dimov. 2020. Lambda2: A C++14 Lambda Library.

<https://www.boost.org/doc/libs/master/libs/lambda2/doc/html/lambda2.html>

[P0060R0] Mathias Gaunard, Dietmar Kühl. 2015-09-18. Function Object-Based Overloading of Operator Dot.

<https://wg21.link/p0060r0>

[P2321R2] Tim Song. 2021-06-11. zip.

<https://wg21.link/p2321r2>

[P2760R0] Barry Revzin. 2023-09-17. A Plan for C++26 Ranges.

<https://wg21.link/p2760r0>

[P2769R1] Ruslan Arutyunyan, Alexey Kukanov. 2023-05-17. `get_element` customization point object.

<https://wg21.link/p2769r1>